8.11 Exercises

Some of the exercises in this chapter were developed for courses on numerical optimal control at the University of Freiburg, Germany. The authors gratefully acknowledge Joel Andersson, Joris Gillis, Sébastien Gros, Dimitris Kouzoupis, Jesus Lago Garcia, Rien Quirynen, Andrea Zanelli, and Mario Zanon for contributions to the formulation of these exercises; as well as Michael Risbeck, Nishith Patel, Douglas Allan, and Travis Arnold for testing and writing solution scripts.

Exercise 8.1: Newton's method for root finding

In this exercise, we experiment with a full-step Newton method and explore the dependence of the iterates on the problem formulation and the initial guess.

(a) Write a computer program that performs Newton iterations in $\mathbb{R}^n$ that takes as inputs a function $F(z)$, its Jacobian $J(z)$, and a starting point $z_{[0]} \in \mathbb{R}^n$. It shall output the first 20 full-step Newton iterations. Test your program with $R(z) = z^{32} - 2$ starting first at $z_{[0]} = 1$ and then at different positive initial guesses. How many iterations do you typically need in order to obtain a solution that is exact up to machine precision?

(b) An equivalent problem to $z^{32} - 2 = 0$ can be obtained by *lifting* it to a higher dimensional space (Albersmeyer and Diehl, 2010), as follows

$$R(z) = \begin{bmatrix} z_2 - z_1^2 \\ z_3 - z_2^2 \\ z_4 - z_3^2 \\ z_5 - z_4^2 \\ 2 - z_5^2 \end{bmatrix}$$

Use your algorithm to implement Newton's method for this lifted problem and start it at $z_{[0]} = [1\ 1\ 1\ 1\ 1]'$ (note that we use square brackets in the index to denote the Newton iteration). Compare the convergence of the iterates for the lifted problem with those of the equivalent unlifted problem from the previous task, initialized at one.

(c) Consider now the root-finding problem $R(z) = 0$ with $R : \mathbb{R} \to \mathbb{R}, R(z) := \tanh(z) - \frac{1}{2}$. Convergence of Newton's method is sensitive to the chosen initial value z_0. Plot $R(z)$ and observe the nonlinearity. Implement Newton's method with full steps for it, and test if it converges or not for different initial values $z_{[0]}$.

(d) Regard the problem of finding a solution to the nonlinear equation system $2x = e^{y/4}$ and $16x^4 + 81y^4 = 4$ in the two variables $x, y \in \mathbb{R}$. Solve it with your implementation of Newton's method using different initial guesses. Does it always converge, and, if it converges, does it always converge to the same solution?

Exercise 8.2: Newton-type methods for a boundary-value problem

Regard the scalar discrete time system

$$x(k+1) = \frac{1}{10}\left(11x(k) + x(k)^2 + u\right), \quad k = 0, \ldots, N-1$$

with boundary conditions

$$x(0) = x_0 \qquad x(N) = 0$$

We fix the initial condition to $x_0 = 0.1$ and the horizon length to $N = 30$. The aim is to find the control value $u \in \mathbb{R}$—which is kept constant over the whole horizon—in order to steer the system to the origin at the final time, i.e., to satisfy the constraint $x(N) = 0$. This is a two-point boundary-value problem (BVP). In this exercise, we formulate this BVP as a root-finding problem in two different ways: first, with the sequential approach, i.e., with only the single control as decision variable; and second, with the simultaneous approach, i.e., with all 31 states plus the control as decision variables.

(a) Formulate and solve the problem with the sequential approach, and solve it with an exact Newton's method initialized at $u = 0$. Plot the state trajectories in each iteration. Also plot the residual values $x(N)$ and the variable u as a function of the Newton iteration index.

(b) Now formulate and solve the problem with the simultaneous approach, and solve it with an exact Newton's method initialized at $u = 0$ and the corresponding state sequence that is obtained by forward simulation started at x_0. Plot the state trajectories in each iteration.

Plot again the residual values $x(N)$ and the variable u as a function of the Newton iteration index, and compare with the results that you have obtained with the sequential approach. Do you observe differences in the convergence speed?

(c) One feature of the simultaneous approach is that its states can be initialized with any trajectory, even an infeasible one. Initialize the simultaneous approach with the all-zero trajectory, and again observe the trajectories and the convergence speed.

(d) Now solve both formulations with a Newton-type method that uses a constant Jacobian. For both approaches, the constant Jacobian corresponds to the exact Jacobian at the solution of the same problem for $x_0 = 0$, where all states and the control are zero. Start with implementing the sequential approach, and initialize the iterates at $u = 0$. Again, plot the residual values $x(N)$ and the variable u as a function of iteration index.

(e) Now implement the simultaneous approach with a fixed Jacobian approximation. Again, the Jacobian approximation corresponds to the exact Jacobian at the solution of the neighboring problem with $x_0 = 0$, i.e., the all zero trajectory. Start the Newton-type iterations with all states and the control set to zero, and plot the residual values $x(N)$ and the variable u as a function of iteration index. Discuss the differences of convergence speed with the sequential approach and with the exact Newton methods from before.

(f) The performance of the sequential approach can be improved if one introduces the initial state $x(0)$ as a second decision variable. This allows more freedom for the initialization, and one can automatically profit from tangential solution

predictors. Adapt your exact Newton method, initialize the problem in the all-zero solution and again observe the results.

(g) If u^* is the exact solution that is found at the end of the iterations, plot the logarithm of $|u - u^*|$ versus the iteration number for all six numerical experiments (a)–(f), and compare.

(h) The linear system that needs to be solved in each iteration of the simultaneous approach is large and sparse. We can use condensing in order to reduce the linear system to size one. Implement a condensing-based linear system solver that only uses multiplications and additions, and one division. Compare the iterations with the full-space linear algebra approach, and discuss the differences in the iterations, if any.

Exercise 8.3: Convex functions

Determine and explain whether the following functions are convex or not on their respective domains.

(a) $f(x) = c'x + x'A'Ax$ on $\mathbb{R}^n$

(b) $f(x) = -c'x - x'A'Ax$ on $\mathbb{R}^n$

(c) $f(x) = \log(c'x) + \exp(b'x)$ on $\{x \in \mathbb{R}^n \mid c'x > 0\}$

(d) $f(x) = -\log(c'x) - \exp(b'x)$ on $\{x \in \mathbb{R}^n \mid c'x > 0\}$

(e) $f(x) = 1/(x_1x_2)$ on $\mathbb{R}^2_{++}$

(f) $f(x) = x_1/x_2$ on $\mathbb{R}^2_{++}$

Exercise 8.4: Convex sets

Determine and explain whether the following sets are convex or not.

(a) A ball, i.e., a set of the form

$$\Omega = \{x \mid |x - x_c| \leq r\}$$

(b) A sublevel set of a convex function $f : \mathbb{R}^n \to \mathbb{R}$ for a constant $c \in \mathbb{R}$

$$\Omega = \{x \in \mathbb{R}^n \mid f(x) \leq c\}$$

(c) A superlevel set of a convex function $f : \mathbb{R}^n \to \mathbb{R}$ for a constant $c \in \mathbb{R}$

$$\Omega = \{x \in \mathbb{R}^n \mid f(x) \geq c\}$$

(d) The set

$$\Omega = \{x \in \mathbb{R}^n \mid x'B'Bx \leq b'x\}$$

(e) The set

$$\Omega = \{x \in \mathbb{R}^n \mid x'B'Bx \geq b'x\}$$

(f) A cone, i.e., a set of the form

$$\Omega = \{(x, \alpha) \in \mathbb{R}^n \times \mathbb{R} \mid |x| \leq \alpha\}$$

(g) A wedge, i.e., a set of the form

$$\{x \in \mathbb{R}^n \mid a_1'x \le b_1, a_2'x \le b_2\}$$

(h) A polyhedron

$$\{x \in \mathbb{R}^n \mid Ax \le b\}$$

(i) The set of points closer to one set than another

$$\Omega = \{x \in \mathbb{R}^n \mid \operatorname{dist}(x, S) \le \operatorname{dist}(x, \mathcal{T})\}$$

where $\operatorname{dist}(x, S) := \inf\{|x - z|_2 \mid z \in S\}$.

Exercise 8.5: Finite differences: theory of optimal perturbation size

Assume we have a twice continuously differentiable function $f : \mathbb{R} \to \mathbb{R}$ and we want to evaluate its derivative $f'(x_0)$ at x_0 with finite differences. Further assume that for all $x \in [x_0 - \delta, x_0 + \delta]$ holds that

$$|f(x)| \le f_{\max} \qquad |f''(x)| \le f''_{\max} \qquad |f'''(x)| \le f'''_{\max}$$

We assume $\delta > t$ for any perturbation size t in the following finite difference approximations. Due to finite machine precision ϵ_{mach} that leads to truncation errors, the computed function $\tilde{f}(x) = f(x)(1 + \epsilon(x))$ is perturbed by noise $\epsilon(x)$ that satisfies the bound

$$|\epsilon(x)| \le \epsilon_{\text{mach}}$$

(a) Compute a bound on the error of the forward difference approximation

$$\tilde{f}'_{\text{fd},t}(x_0) := \frac{\tilde{f}(x_0 + t) - \tilde{f}(x_0)}{t}$$

namely, a function $\psi(t; f_{\max}, f''_{\max}, \epsilon_{\text{mach}})$ that satisfies

$$\left|\tilde{f}'_{\text{fd},t}(x_0) - f'(x_0)\right| \le \psi(t; f_{\max}, f''_{\max}, \epsilon_{\text{mach}})$$

(b) Which value t^* minimizes this bound and which value ψ^* has the bound at t^*?

(c) Perform a similar error analysis for the central difference quotient

$$\tilde{f}'_{\text{cd},t}(x_0) := \frac{\tilde{f}(x_0 + t) - \tilde{f}(x_0 - t)}{2t}$$

that is, compute a bound

$$\left|\tilde{f}'_{\text{fd},t}(x_0) - f'(x_0)\right| \le \psi_{\text{cd}}(t; f_{\max}, f''_{\max}, f'''_{\max}, \epsilon_{\text{mach}})$$

(d) For central differences, what is the optimal perturbation size t^*_{cd} and what is the size ψ^*_{cd} of the resulting bound on the error?

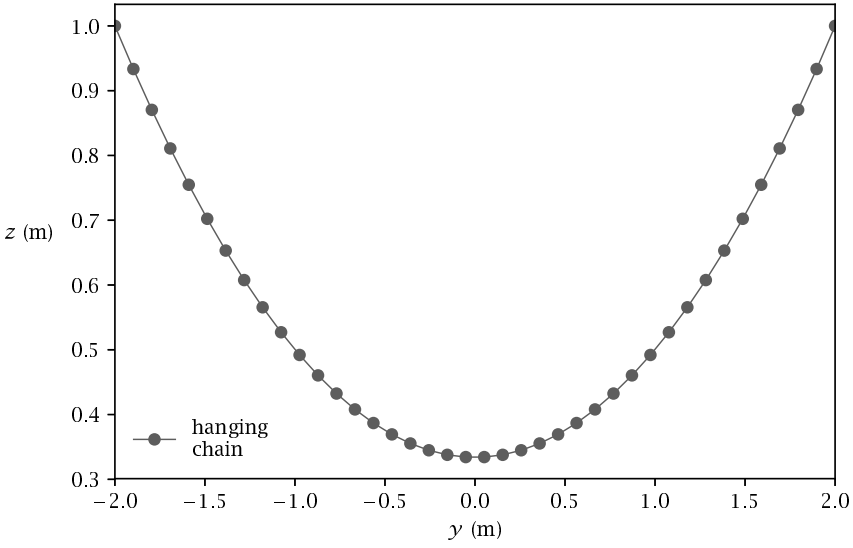

Figure 8.7: A hanging chain at rest. See Exercise 8.6(b).

Exercise 8.6: Finding the equilibrium point of a hanging chain using CasADi

Consider an elastic chain attached to two supports and hanging in-between. Let us discretize it with N mass points connected by $N-1$ springs. Each mass i has position (y_i, z_i), $i = 1, \ldots, N$.

Our task is to minimize the total potential energy, which is made up by potential energy in each string and potential energy of each mass according to

$$J(y_1, z_1, \ldots, y_n, z_n) = \underbrace{\frac{1}{2}\sum_{i=1}^{N-1} D_i\left((y_i - y_{i+1})^2 + (z_i - z_{i+1})^2\right)}_{\text{spring potential energy}} + \underbrace{g_0 \sum_{i=1}^{N} m_i z_i}_{\text{gravitational potential energy}} \tag{8.61}$$

subject to constraints modeling the ground.

(a) CasADi is an open-source software tool for solving optimization problems in general and optimal control problems (OCPs) in particular. In its most typical usage, it is used to formulate and solve constrained optimization problems of the form

$$\begin{aligned} \underset{x}{\text{minimize}} \quad & f(x) \\ \text{subject to} \quad & \underline{x} \le x \le \overline{x} \\ & \underline{g} \le g(x) \le \overline{g} \end{aligned} \tag{8.62}$$

where $x \in \mathbb{R}^{n_x}$ is the decision variable, $f : \mathbb{R}^{n_x} \to \mathbb{R}$ is the objective function, and $g : \mathbb{R}^{n_x} \to \mathbb{R}^{n_g}$ is the constraint function. For equality constraints, the

upper and lower bounds are equal.

If you have not already done so, go to `casadi.org` and locate the installation instructions. On most platforms, installing CasADi amounts to downloading a binary installation and placing it somewhere in your path. Version 3.3 of CasADi on Octave/MATLAB was used in this edition, so make sure that you are not using a version older than this and keep an eye on the text website for incompatibilities with future versions of CasADi. Locate the CasADi user guide and, with an Octave or MATLAB interpreter in front of you, read Chapters 1 through 4. These chapters give you an overview of the scope and syntax of CasADi.

(b) We assume that f is a convex quadratic function and g is a linear function. In this case we refer to (8.62) as a convex quadratic program (QP). To solve a QP with CasADi, we construct symbolic expressions for x, f, and g, and use this to construct a solver object that can be called one or more times with different values for $\overline{x}$, $\underline{x}$, $\overline{g}$, and $\underline{g}$. An initial guess for x can also be provided, but this is less important for convex QPs, where the solution is unique.

Figure 8.7 shows the solution of the unconstrained problem using the open-source QP solver qpOASES with $N = 40$, $m_i = 40/N$ kg, $D_i = 70N$ N/m, and $g_0 = 9.81$ m/s^2. The first and last mass points are fixed to $(-2, 1)$ and $(2, 1)$, respectively. Go through the code for the figure and make sure you understand the steps.

(c) Now introduce ground constraints: $z_i \geq 0.5$ and $z_i \geq 0.5 + 0.1\,y_i$, for $i = 2, \cdots, N-2$. Resolve the QP and compare with the unconstrained solution.

(d) We now want to formulate and solve a nonlinear program (NLP). Since an NLP is a generalization of a QP, we can solve the above problem with an NLP solver. This can be done by simply changing `casadi.qpsol` in the script to `casadi.nlpsol` and the solver plugin `'qpoases'` with `'ipopt'`, corresponding to the open-source NLP solver IPOPT. Are the solutions of the NLP and QP solver the same?

(e) Now, replace the linear equalities by nonlinear ones that are given by $z_i \geq 0.5 + 0.1\,y_i^2$ for $i = 2, \cdots, N-2$. Modify the expressions from before to formulate and solve the NLP, and visualize the solution. Is the NLP convex?

(f) Now, by modifications of the expressions from before, formulate and solve an NLP where the inequality constraints are replaced by $z_i \geq 0.8 + 0.05\,y_i - 0.1\,y_i^2$ for $i = 2, \cdots, N-2$. Is this NLP convex?

Exercise 8.7: Direct single shooting versus direct multiple shooting

Consider the following OCP, corresponding to driving a Van der Pol oscillator to the origin, on a time horizon with length $T = 10$

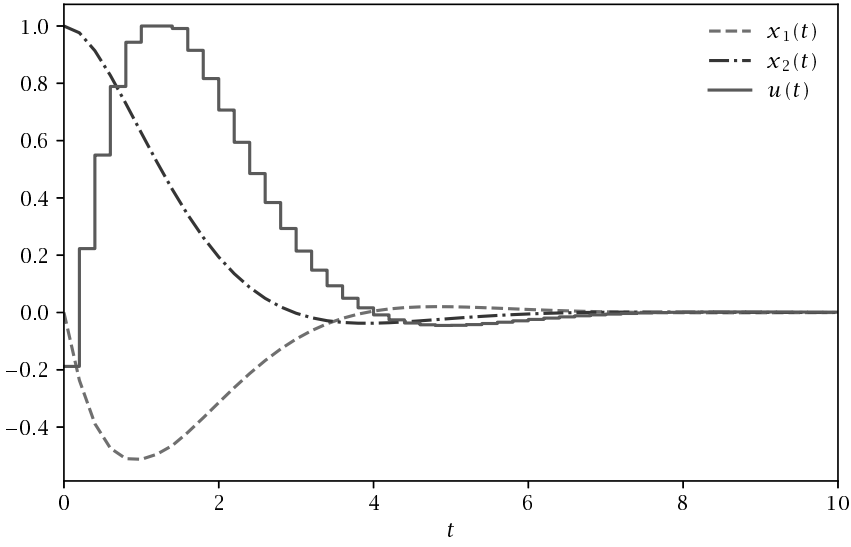

Figure 8.8: Direct single shooting solution for (8.63) without path constraints.

$$
\begin{aligned}
\underset{x(\cdot),u(\cdot)}{\text{minimize}} \quad & \int_0^T (x_1(t)^2 + x_2(t)^2 + u(t)^2)\, dt \\
\text{subject to} \quad & \dot{x}_1(t) = (1 - x_2(t)^2)\, x_1(t) - x_2(t) + u(t) \\
& \dot{x}_2(t) = x_1(t) \\
& -1 \le u(t) \le 1, \quad t \in [0, T] \\
& x_1(0) = 0, \quad x_1(T) = 0 \\
& x_2(0) = 1, \quad x_2(T) = 0 \\
& -0.25 \le x_1(t), \quad t \in [0, T]
\end{aligned}
\tag{8.63}
$$

We will solve this problem using direct single shooting and direct multiple shooting using IPOPT as the NLP solver.

(a) Figure 8.8 shows the solution to the above problem using a direct single shooting approach, without enforcing the constraint $-0.25 \le x_1(t)$. Go through the code for the figure step by step. The code begins with a modeling step, where symbolic expressions for the continuous-time model are constructed. Thereafter, the problem is transformed into discrete time by formulating an object that integrates the system forward in time using a single step of the RK4 method. This function also calculates the contribution to the objective function for the same interval using the same integrator method. In the next part of the code, a

symbolic representation of the NLP is constructed, starting with empty lists of variables and constraints. This symbolic representation of the NLP is used to define an NLP solver object using IPOPT as the underlying solver. Finally, the solver object is evaluated to obtain the optimal solution.

(b) Modify the code so that the path constraint on $x_1(t)$ is being respected. You only need to enforce this constraint at the end of each control interval. This should result in additional components to the NLP constraint function $G(w)$, which will now have upper and lower bounds similar to the decision variable w. Resolve the modified problem and compare the solution.

(c) Modify the code to implement the direct multiple-shooting method instead of direct single shooting. This means introducing decision variables corresponding to not only the control trajectory, but also the state trajectory. The added decision variables will be matched with an equal number of new equality constraints, enforcing that the NLP solution corresponds to a continuous state trajectory. The initial and terminal conditions on the state can be formulated as upper and lower bounds on the corresponding elements of w. Use $x(t) = 0$ as the initial guess for the state trajectory.

(d) Compare the IPOPT output for both transcriptions. How did the change from direct single shooting to direct multiple shooting influence

- The number of iterations?
- The number of nonzeros in the Jacobian of the constraints?
- The number of nonzeros in the Hessian of the Lagrangian?

(e) Generalize the RK4 method so that it takes $M = 4$ steps instead of just one. This corresponds to a higher-accuracy integration of the model dynamics. Approximately how much smaller discretization error can we expect from this change?

(f) Replace the RK4 integrator with the variable-order, variable-step size code CVODES from the SUNDIALS suite, available as the `'cvodes'` plugin for `casadi.integrator`. Use 10^{-8} for the relative and absolute tolerances. Consult CasADi's user guide for syntax. What are the advantages and disadvantages of using this integrator over the fixed-step RK4 method used until now?

Exercise 8.8: Direct collocation

Collocation, in its most basic sense, refers to a way of solving initial-value problems by approximating the state trajectory with piecewise polynomials. For each step of the integrator, corresponding to an interval of time, we choose the coefficients of these polynomials to ensure that the ODE becomes exactly satisfied at a given set of time points. In the following, we choose the *Gauss-Legendre* collocation integrator of sixth order, which has $d = 3$ collocation points. Together with the point 0 at the start of the interval $[0, 1]$, we have four time points to define the collocation polynomial

$$\tau_0 = 0 \qquad \tau_1 = 1/2 - \sqrt{15}/10 \qquad \tau_2 = 1/2 \qquad \tau_3 = 1/2 + \sqrt{15}/10$$

Using these time points, we define the corresponding Lagrange polynomials

$$L_j(\tau) = \prod_{r=0, r\neq j}^{d} \frac{\tau - \tau_r}{\tau_j - \tau_r}$$

Introducing a uniform time grid $t_k = k\,h$, $k = 0, \ldots, N$, with the corresponding state values $x_k := x(t_k)$, we can approximate the state trajectory inside each interval $[t_k, t_{k+1}]$ as a linear combination of these basis functions

$$\tilde{x}_k(t) = \sum_{r=0}^{d} L_r\left(\frac{t - t_k}{h}\right) x_{k,r}$$

By differentiation, we get an approximation of the time derivative at each collocation point for $j = 1, \ldots, 3$

$$\dot{\tilde{x}}_k(t_{k,j}) = \frac{1}{h} \sum_{r=0}^{d} \dot{L}_r(\tau_j)\, x_{k,r} := \frac{1}{h} \sum_{r=0}^{d} C_{r,j}\, x_{k,r}$$

We also can get an expression for the state at the end of the interval

$$\tilde{x}_{k+1,0} = \sum_{r=0}^{d} L_r(1)\, x_{k,r} := \sum_{r=0}^{d} D_r\, x_{k,r}$$

Finally, we also can integrate our approximation over the interval, giving a formula for *quadratures*

$$\int_{t_k}^{t_{k+1}} \tilde{x}_k(t)\, dt = h \sum_{r=0}^{d} \int_0^1 L_r(\tau)\, d\tau\, x_{k,r} := h \sum_{r=1}^{d} b_r\, x_{k,r}$$

(a) Figure 8.9 shows an open-loop simulation for the ODE in (8.63) using Gauss-Legendre collocation of order 2, 4, and 6. A constant control $u(t) = 0.5$ was applied and the initial conditions were given by $x(0) = [0, 1]'$. The figure on the left shows the first state $x_1(t)$ for the three methods as well as a high-accuracy solution obtained from CVODES, which uses a backward differentiation formula (BDF) method. In the figure on the right we see the discretization error, as compared with CVODES. Go through the code for the figure and make sure you understand it. Using this script as a template, replace the integrator in the direct multiple-shooting method from Exercise 8.7 with this collocation integrator. Make sure that you obtain the same solution. The structure of the NLP should remain unchanged—you are still implementing the direct multiple-shooting approach, only with a different integrator method.

(b) In the NLP transcription step, replace the embedded function call with additional degrees of freedom corresponding to the state at all the collocation points. Enforce the collocation equations at the NLP level instead of the integrator level.

Enforce upper and lower bounds on the state at all collocation points. Compare the solution time and number of nonzeros in the Jacobian and Hessian matrices with the direct multiple-shooting method.

Exercise 8.9: Gauss-Newton SQP iterations for optimal control

Consider a nonlinear pendulum defined by

$$\dot{x}(t) = f(x(t), u(t)) = \begin{bmatrix} v(t) \\ -C \sin(p(t)/C) \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u(t)$$

with state $x = [p, v]'$ and $C := 180/\pi/10$, to solve an OCP using a direct multiple-shooting method and a self-written sequential quadratic programming (SQP) solver with a Gauss-Newton Hessian.

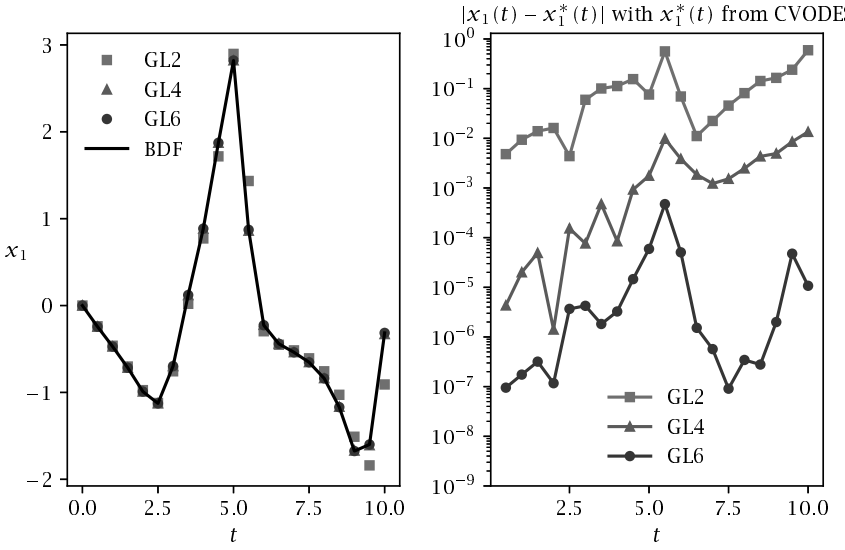

Figure 8.9: Open-loop simulation for (8.63) using collocation.

(a) Starting with the pendulum at $\bar{x}_0 = [10\ 0]'$, we aim to minimize the required controls to bring the pendulum to $x_N = [0\ 0]'$ in a time horizon $T = 10\,\text{s}$. Regarding bounds on p, v, and u, namely $p_{\max} = 10$, $v_{\max} = 10$, and $u_{\max} = 3$, the required controls can be obtained as the solution of the following OCP

$$
\begin{aligned}
\underset{\substack{x_0,u_0,x_1,\ldots,\\ u_{N-1},x_N}}{\text{minimize}} \quad & \frac{1}{2}\sum_{k=0}^{N-1} |u_k|_2^2 \\
\text{subject to} \quad & \bar{x}_0 - x_0 = 0 \\
& \Phi(x_k, u_k) - x_{k+1} = 0, \quad k = 0,\ldots,N-1 \\
& x_N = 0 \\
& -x_{\max} \le x_k \le x_{\max}, \quad k = 0,\ldots,N-1 \\
& -u_{\max} \le u_k \le u_{\max}, \quad k = 0,\ldots,N-1
\end{aligned}
$$

Formulate the discrete dynamics $x_{k+1} = \Phi(x_k, u_k)$ using a RK4 integrator with a time step $\Delta t = 0.2\,\text{s}$. Encapsulate the code in a single CasADi function of the form of a CasADi function object as in Exercise 8.7. Simulate the system forward in time and plot the result.

(b) Using $w = (x_0, u_0, \ldots, u_{N-1}, x_N)$ as the NLP decision variable, we can formulate the equality constraint function $G(w)$, the least squares function $M(w)$, and the

bounds vector $w_{\max}$ so that the above OCP can be written

$$\begin{aligned} \underset{w}{\text{minimize}} \quad & \frac{1}{2}|M(w)|_2^2 \\ \text{subject to} \quad & G(w) = 0 \\ & -w_{\max} \le w \le w_{\max} \end{aligned}$$

The SQP method with Gauss-Newton Hessian solves a linearized version of this problem in each iteration. More specifically, if the current iterate is $\bar{w}$, the next iterate is given by $\bar{w} + \Delta w$, where Δw is the solution of the following QP

$$\begin{aligned} \underset{\Delta w}{\text{minimize}} \quad & \frac{1}{2}\Delta w' J_M(\bar{w})' J_M(\bar{w})\Delta w + M(\bar{w})' J_M(\bar{w})\Delta w \\ \text{subject to} \quad & G(\bar{w}) + J_G(\bar{w})\Delta w = 0 \\ & -w_{\max} - \bar{w} \le \Delta w \le w_{\max} - \bar{w} \end{aligned} \tag{8.64}$$

In order to implement the Gauss-Newton method, we need the Jacobians $J_G(w) = \frac{\partial G}{\partial w}(w)$ and $J_M(w) = \frac{\partial M}{\partial w}(w)$, both of which can be efficiently obtained using CasADi's `jacobian` command. In this case the Gauss-Newton Hessian $H = J_M(\bar{w})' J_M(\bar{w})$ can readily be obtained by pen and paper. Define what H_x and H_u need to be in the Hessian

$$H = \begin{bmatrix} H_x & & & \\ & H_u & & \\ & & \ddots & \\ & & & H_x \end{bmatrix} \qquad H_x = \begin{bmatrix} & \end{bmatrix} \qquad H_u = \begin{bmatrix} & \end{bmatrix}$$

(c) Figure 8.10 shows the control trajectory after 0, 1, 2, and 6 iterations of the Gauss-Newton method applied to a direct multiple-shooting transcription of (8.63). Go through the code for the figure step by step. You should recognize much of the code from the solution to Exercise 8.7. The code represents a simplified, yet efficient way of using CasADi to solve OCPs.

Modify the code to solve the pendulum problem. Note that the sparsity patterns of the linear and quadratic terms of the QP are printed out at the beginning of the execution. $J_G(w)$ is a block sparse matrix with blocks being either identity matrices I or partial derivatives $A_k = \frac{\partial \Phi}{\partial x}(x_k, u_k)$ and $B_k = \frac{\partial \Phi}{\partial u}(x_k, u_k)$.

Initialize the Gauss-Newton procedure at $w = 0$, and stop the iterations when $|w_{k+1} - w_k|$ gets smaller than 10^{-4}. Plot the iterates as well as the vector G during the iterations. How many iterations do you need?

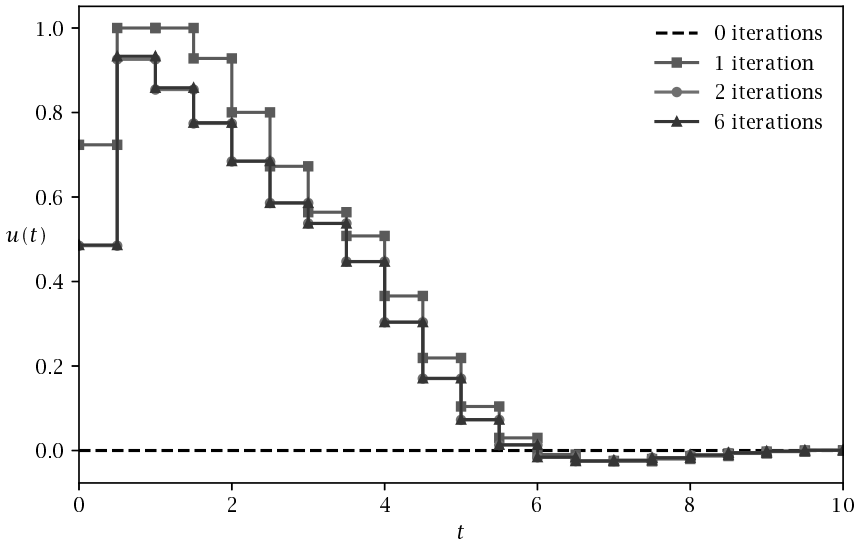

Figure 8.10: Gauss-Newton iterations for a direct multiple-shooting transcription of (8.63); $u(t)$ after 0, 1, 2, and 6 Gauss-Newton iterations.

Exercise 8.10: Real-time iterations and nonlinear MPC

We return to the OCP from Exercise 8.9

$$
\begin{aligned}
\underset{\substack{x_0,u_0,x_1,\ldots,\\ u_{N-1},x_N}}{\text{minimize}} \quad & \frac{1}{2}\sum_{k=0}^{N-1} |\, u_k \,|_2^2 \\
\text{subject to} \quad & \bar{x}_0 - x_0 = 0 \\
& \Phi(x_k, u_k) - x_{k+1} = 0, \qquad k = 0, \ldots, N-1 \\
& x_N = 0 \\
& -x_{\max} \leq x_k \leq x_{\max}, \quad k = 0, \ldots, N-1 \\
& -u_{\max} \leq u_k \leq u_{\max}, \quad k = 0, \ldots, N-1
\end{aligned}
$$

In this problem, we regard $\bar{x}_0$ as a parameter and modify the simultaneous Gauss-Newton algorithm from Exercise 8.9. In particular, we modify this algorithm to perform real-time iterations for different values of $\bar{x}_0$, so that we can use the algorithm to perform closed-loop nonlinear MPC simulations for stabilization of the nonlinear pendulum.

(a) Modify the function `sqpstep` from the solution of Exercise 8.9 so that it accepts the parameter $\bar{x}_0$. You would need to update the upper and lower bounds on w accordingly. Test it and make sure that it works.

(b) In order to visualize the generalized tangential predictor, call the `sqpstep` method with different values for $\bar{x}_0$ while resetting the variable vector $\bar{w}$ to its initial value (zero) between each call. Use a linear interpolation for $\bar{x}_0$ with 100 points between zero and the value $(10, 0)'$, i.e., set $\bar{x}_0 = \lambda[10\ 0]'$ for $\lambda \in [0, 1]$. Plot the first control u_0 as a function of λ and keep your plot.

(c) To compute the exact solution manifold with relatively high accuracy, perform now the same procedure for the same 100 increasing values of λ, but this time perform for each value of λ multiple Gauss-Newton iterations, i.e., replace each call to `sqpstep` with, e.g., 10 calls without changing $\bar{x}_0$. Plot the obtained values for u_0 and compare with the tangential predictor from the previous task by plotting them in the same plot.

(d) In order to see how the real-time iterations work in a more realistic setting, let the values of λ jump faster from 0 to 1, e.g., by doing only 10 steps, and plot the result again into the same plot.

(e) Modify the previous algorithm as follows: after each change of λ by 0.1, keep it constant for nine iterations, before you do the next jump. This results in about 100 consecutive real-time iterations. Interpret what you see.

(f) Now we do the first *closed-loop simulation*: set the value of $\bar{x}_0^{[1]}$ to $[10\ 0]'$ and initialize $w^{[0]}$ at zero, and perform the first real-time iteration by calling `sqpstep`. This iteration yields the new solution guess $w^{[1]}$ and corresponding control $u_0^{[1]}$. Use this control at the "real plant," i.e., generate the next value of $\bar{x}_0$, which we denote $\bar{x}_0^{[2]}$, by calling the one-step simulation function, $\bar{x}_0^{[2]} := \Phi(\bar{x}_0^{[1]}, u_0^{[1]})$. Close the loop by calling `sqpstep` using $w^{[1]}$ and $\bar{x}_0^{[2]}$, etc., and perform 100 iterations. For better observation, plot after each real-time iteration the control and state variables on the whole prediction horizon. (It is interesting to note that the state trajectory is not necessarily feasible).

Also observe what happens with the states $\bar{x}_0$ during the scenario, and plot them in another plot against the time index. Do they converge, and if yes, to what value?

(g) Now we make the control problem more difficult by treating the pendulum in an upright position, which is unstable. This is simply done by changing the sign in front of the sine in the differential equation, i.e., our model is now

$$f(x(t), u(t)) = \begin{bmatrix} v(t) \\ C\sin(p(t)/C) \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u(t) \tag{8.65}$$

Start your real-time iterations again at $w^{[0]} = 0$ and set $\bar{x}_0^{[1]}$ to $[10\ 0]'$, and perform the same closed-loop simulation as before. Explain what happens.

Bibliography

J. Albersmeyer. *Adjoint-based algorithms and numerical methods for sensitivity generation and optimization of large scale dynamic systems*. PhD thesis, University of Heidelberg, 2010.

J. Albersmeyer and M. Diehl. The lifted Newton method and its application in optimization. *SIAM Journal on Optimization*, 20(3):1655–1684, 2010.

J. Andersson. *A General-Purpose Software Framework for Dynamic Optimization*. PhD thesis, KU Leuven, October 2013.

J. Andersson, J. Akesson, and M. Diehl. CasADi – a symbolic package for automatic differentiation and optimal control. In *Recent Advances in Algorithmic Differentiation*, volume 87 of *Lecture Notes in Computational Science and Engineering*, pages 297–307. Springer, 2012.

U. M. Ascher and L. R. Petzold. *Computer Methods for Ordinary Differential Equations and Differential-Algebraic Equations*. SIAM, Philadelphia, 1998.

D. Axehill. Controlling the level of sparsity in MPC. *Systems & Control Letters*, 76:1–7, 2015.

D. Axehill and M. Morari. An alternative use of the Riccati recursion for efficient optimization. *Systems & Control Letters*, 61(1):37–40, 2012.

I. Bauer, H. G. Bock, S. Körkel, and J. P. Schlöder. Numerical methods for optimum experimental design in DAE systems. *J. Comput. Appl. Math.*, 120 (1-2):1–15, 2000.

A. Ben-Tal and A. Nemirovski. *Lectures on Modern Convex Optimization: Analysis, Algorithms, and Engineering Applications*. MPS-SIAM Series on Optimization. MPS-SIAM, Philadelphia, 2001.

D. P. Bertsekas. *Nonlinear Programming*. Athena Scientific, Belmont, MA, second edition, 1999.

J. T. Betts. *Practical Methods for Optimal Control Using Nonlinear Programming*. SIAM, Philadelphia, 2001.

L. T. Biegler. *Nonlinear Programming*. MOS-SIAM Series on Optimization. SIAM, 2010.

T. Binder, L. Blank, H. G. Bock, R. Bulirsch, W. Dahmen, M. Diehl, T. Kronseder, W. Marquardt, J. P. Schlöder, and O. V. Stryk. Introduction to model based optimization of chemical processes on moving horizons. *Online Optimization of Large Scale Systems: State of the Art, Springer*, pages 295–340, 2001.

H. G. Bock. Numerical treatment of inverse problems in chemical reaction kinetics. In K. H. Ebert, P. Deuflhard, and W. Jäger, editors, *Modelling of Chemical Reaction Systems*, volume 18 of *Springer Series in Chemical Physics*, pages 102–125. Springer, Heidelberg, 1981.

H. G. Bock. Recent advances in parameter identification techniques for ODE. In *Numerical Treatment of Inverse Problems in Differential and Integral Equations*, pages 95–121. Birkhäuser, 1983.

H. G. Bock and K. J. Plitt. A multiple shooting algorithm for direct solution of optimal control problems. In *Proceedings of the IFAC World Congress*, pages 242–247. Pergamon Press, 1984.

S. P. Boyd and L. Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.

K. E. Brenan, S. L. Campbell, and L. R. Petzold. *Numerical solution of initial-value problems in differential-algebraic equations*. Classics in Applied Mathematics 14. SIAM, Philadelphia, 1996.

A. E. Bryson and Y. Ho. *Applied Optimal Control*. Hemisphere Publishing, New York, 1975.

A. R. Curtis, M. J. D. Powell, and J. K. Reid. On the estimation of sparse Jacobian matrices. *J. Inst. Math. Appl.*, 13:117–119, 1974.

P. Deuflhard. *Newton methods for nonlinear problems: affine invariance and adaptive algorithms*, volume 35. Springer, 2011.

M. Diehl. *Real-Time Optimization for Large Scale Nonlinear Processes*. PhD thesis, Universität Heidelberg, 2001.

M. Diehl, H. G. Bock, J. P. Schlöder, R. Findeisen, Z. Nagy, and F. Allgöwer. Real-time optimization and nonlinear model predictive control of processes governed by differential-algebraic equations. *J. Proc. Cont.*, 12(4):577–585, 2002.

M. Diehl, H. J. Ferreau, and N. Haverbeke. Efficient numerical methods for nonlinear MPC and moving horizon estimation. In L. Magni, M. D. Raimondo, and F. Allgöwer, editors, *Nonlinear model predictive control*, volume 384 of *Lecture Notes in Control and Information Sciences*, pages 391–417. Springer, 2009.

A. Domahidi. *Methods and Tools for Embedded Optimization and Control*. PhD thesis, ETH Zürich, 2013.

H. J. Ferreau, C. Kirches, A. Potschka, H. G. Bock, and M. Diehl. qpOASES: a parametric active-set algorithm for quadratic programming. *Mathematical Programming Computation*, 6(4):327–363, 2014.

R. Franke. *Integrierte dynamische Modellierung und Optimierung von Systemen mit saisonaler Wärmespeicherung*. PhD thesis, Technische Universität Ilmenau, Germany, 1998.

J. V. Frasch, S. Sager, and M. Diehl. A parallel quadratic programming method for dynamic optimization problems. *Mathematical Programming Computations*, 7(3):289–329, 2015.

G. Frison. *Algorithms and Methods for High-Performance Model Predictive Control*. PhD thesis, Technical University of Denmark (DTU), 2015.

A. H. Gebremedhin, F. Manne, and A. Pothen. What color is your Jacobian? Graph coloring for computing derivatives. *SIAM Review*, 47:629–705, 2005.

M. Gerdts. *Optimal Control of ODEs and DAEs*. Berlin, Boston: De Gruyter, 2011.

P. Gill, W. Murray, and M. Saunders. SNOPT: An SQP Algorithm for Large-Scale Constrained Optimization. *SIAM Review*, 47(1):99–131, 2005.

J. Gillis. *Practical methods for approximate robust periodic optimal control of nonlinear mechanical systems*. PhD thesis, KU Leuven, 2015.

A. Griewank and A. Walther. *Evaluating Derivatives*. SIAM, 2 edition, 2008.

S. Gros and M. Diehl. *Numerical Optimal Control*. 2018. (In preparation).

J. Guddat, F. G. Vasquez, and H. T. Jongen. *Parametric Optimization: Singularities, Pathfollowing and Jumps*. Teubner, Stuttgart, 1990.

E. Hairer, S. P. Nørsett, and G. Wanner. *Solving Ordinary Differential Equations I*. Springer Series in Computational Mathematics. Springer, Berlin, 2nd edition, 1993.

E. Hairer, S. P. Nørsett, and G. Wanner. *Solving Ordinary Differential Equations II – Stiff and Differential-Algebraic Problems*. Springer Series in Computational Mathematics. Springer, Berlin, 2nd edition, 1996.

A. C. Hindmarsh, P. N. Brown, K. E. Grant, S. L. Lee, R. Serban, D. E. Shumaker, and C. S. Woodward. SUNDIALS: Suite of Nonlinear and Differential/Algebraic Equation Solvers. *ACM Trans. Math. Soft.*, 31:363–396, 2005.

B. Houska, H. J. Ferreau, and M. Diehl. ACADO toolkit – an open source framework for automatic control and dynamic optimization. *Optimal Cont. Appl. Meth.*, 32(3):298–312, 2011.

D. H. Jacobson and D. Q. Mayne. *Differential dynamic programming*, volume 24 of *Modern Analytic and Computational Methods in Science and Mathematics*. American Elsevier Pub. Co., 1970.

M. R. Kristensen, J. B. Jørgensen, P. G. Thomsen, and S. B. Jørgensen. An ESDIRK method with sensitivity analysis capabilities. *Computers and Chemical Engineering*, 28:2695–2707, 2004.

D. B. Leineweber, I. Bauer, A. A. S. Schäfer, H. G. Bock, and J. P. Schlöder. An Efficient Multiple Shooting Based Reduced SQP Strategy for Large-Scale Dynamic Process Optimization (Parts I and II). *Comput. Chem. Eng.*, 27: 157–174, 2003.

W. Li and E. Todorov. Iterative linear quadratic regulator design for nonlinear biological movement systems. In *Proceedings of the 1st International Conference on Informatics in Control, Automation and Robotics*, 2004.

W. C. Li and L. T. Biegler. Multistep, Newton-Type Control Strategies for Constrained Nonlinear Processes. *Chem. Eng. Res. Des.*, 67:562–577, 1989.

D. Q. Mayne. A second-order gradient method for determining optimal trajectories of non-linear discrete-time systems. *Int. J. Control*, 3(1):85–96, 1966.

Y. Nesterov. *Introductory lectures on convex optimization: a basic course*, volume 87 of *Applied Optimization*. Kluwer Academic Publishers, 2004.

J. Nocedal and S. J. Wright. *Numerical Optimization*. Springer, New York, second edition, 2006.

T. Ohtsuka. A continuation/GMRES method for fast computation of nonlinear receding horizon control. *Automatica*, 40(4):563–574, 2004.

G. Pannocchia, J. B. Rawlings, D. Q. Mayne, and G. Mancuso. Whither discrete time model predictive control? *IEEE Trans. Auto. Cont.*, 60(1):246–252, January 2015.

L. R. Petzold, S. Li, Y. Cao, and R. Serban. Sensitivity analysis of differential-algebraic equations and partial differential equations. *Computers and Chemical Engineering*, 30:1553–1559, 2006.

R. Quirynen. *Numerical Simulation Methods for Embedded Optimization*. PhD thesis, KU Leuven and University of Freiburg, 2017.

R. Quirynen, S. Gros, B. Houska, and M. Diehl. Lifted collocation integrators for direct optimal control in ACADO toolkit. *Mathematical Programming Computation*, pages 1–45, 2017a.

R. Quirynen, B. Houska, and M. Diehl. Efficient symmetric Hessian propagation for direct optimal control. *Journal of Process Control*, 50:19–28, 2017b.

S. M. Robinson. Strongly Regular Generalized Equations. *Mathematics of Operations Research, Vol. 5, No. 1 (Feb., 1980), pp. 43-62*, 5:43–62, 1980.

A. Sideris and J. Bobrow. An efficient sequential linear quadratic algorithm for solving unconstrained nonlinear optimal control problems. *IEEE Transactions on Automatic Control*, 50(12):2043–2047, 2005.

M. J. Tenny, S. J. Wright, and J. B. Rawlings. Nonlinear Model Predictive Control via Feasibility-Perturbed Sequential Quadratic Programming. *Comp. Optim. Appl.*, 28:87–121, 2004.

Q. Tran-Dinh, C. Savorgnan, and M. Diehl. Adjoint-based predictor-corrector sequential convex programming for parametric nonlinear optimization. *SIAM J. Optimization*, 22(4):1258–1284, 2012.

C. F. Van Loan. Computing integrals involving the matrix exponential. *IEEE Trans. Automat. Control*, 23(3):395–404, 1978.

A. Wächter and L. T. Biegler. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Math. Prog.*, 106(1):25–57, 2006.

A. Zanelli, A. Domahidi, J. Jerez, and M. Morari. FORCES NLP: An efficient implementation of interior-point methods for multistage nonlinear nonconvex programs. *International Journal of Control*, 2017.